Kaggle TensorFlow Speech Recognition Challenge

Paweł Pozorski[1], *Paweł Florek*[1]

[1] Warsaw University of Technology, Poland

{pawel.pozorski, pawel.florek}.stud@pw.edu.pl

Abstract

Keyword spotting (KWS) is a core component of always-on speech interfaces, where both robustness and low-latency inference are required. This paper presents a controlled comparative study of KWS architectures on the Kaggle TensorFlow Speech Recognition Challenge corpus under a unified training and evaluation protocol. We formulate a fixed 12-class task (10 commands plus __unknown__ and __silence__) and investigate 3 factors that govern performance: front-end representation, architectural inductive bias, and non-command modeling strategy. To isolate architectural effects, all systems are trained from random initialization with identical optimization controls, deterministic settings, and multi-seed evaluation. The experimental design follows a 4-phase selection procedure: feature ablation, global optimization tuning, backbone comparison across model families, and targeted non-command methodology analysis. Final held-out testing is restricted to systems selected in the last phase, including both single-model and within-method ensemble variants. This manuscript reports methodology and evaluation criteria; quantitative outcomes are deferred to a full-results version.

Index Terms: keyword spotting, Kaggle TensorFlow Speech Recognition Challenge, MLOps, convolutional neural networks, transformers, state-space models, recurrent models

1. Introduction and Motivation

Detecting a fixed, limited vocabulary within an audio stream occupies a unique and highly resource-constrained niche in modern speech processing [1, 2]. Recent progress has diversified the model landscape, but many comparisons still confound architectural differences with inconsistent preprocessing and optimization choices. Our objective is to evaluate architectures under a common, from-scratch training protocol, ensuring that observed differences are attributable to structural inductive biases rather than transfer-learning advantages [3].

Our study is structured around three primary research questions, designed to systematically isolate the drivers of KWS efficacy:

- **Q1 (Feature strategy):** Which feature extraction pipeline produces the most generalizable representations for a fixed 1-second classification task? This evaluates whether the default reliance on standard log-mel spectrograms is genuinely optimal.
- **Q2 (Architectural inductive bias):** Which structural prior best captures the temporal-spectral dynamics of brief KWS inputs? We directly compare local spatial hierarchies, global all-to-all attention, and finite-memory sequential compression.
- **Q3 (Non-command class methodology):** Which methodology best recognizes __unknown__ and __silence__ while preserving macro-F1 on the core command classes?

To prevent hyperparameter explosion and to contextualize model evaluation within a practical engineering framework, we structure our experiments into a strict 4-phase pipeline (introduced in Section 3). In this manuscript version, the emphasis is methodological: the Results section specifies the held-out test protocol, while empirical comparisons are reported after all runs complete.

2. Dataset and Task Formulation

To provide a direct evaluation of raw architectural capacity, this project leverages the Kaggle TensorFlow Speech Recognition Challenge dataset (derived from Google Speech Commands v1) [4] as a controlled benchmark. These constraints remove substantial domain-level complexities that typically confound architectural comparisons.

We use a fixed 12-class taxonomy: the target commands are {yes, no, up, down, left, right, on, off, stop, go}, __silence__ is mapped from _background_noise_, and every other spoken label is merged into __unknown__. Dataset preprocessing is standardized as follows: (i) long background-noise recordings are split into one-second silence samples, (ii) short utterances are zero-padded to one second, (iii) samples longer than one second are excluded in strict one-second dataset modes, and (iv) waveform loading enforces a fixed one-second input window for every sample (with padding or truncation as required). Consequently, all downstream feature extraction and modeling stages operate on a uniform temporal support.

Split construction follows three policies. First, command-only small splits are built with fixed per-class quotas (1,000 train, 250 validation, 250 test for each of the 10 commands), and are used in Phases 1–3. Second, extended splits for Phase 4 include all 12 classes and are balanced at the command-vs-unknown level per split. Third, unknown examples used in extended splits are deduplicated by audio-content hash and additionally capped to at most 20,000 samples per split to avoid unknown-class dominance.

Table 1 reports exact split sizes after applying this 12-class remapping to the repository split lists. We provide total sample count, active class count, command/non-command composition, and the intended usage of each split in the experimental pipeline.

2.1. Unknown-Class Synthesis Methodology

Before merging labels, we persist a metadata artifact train/split_lists/unknown_origin_labels.csv

Table 1: *Dataset split statistics under the* 12*-class mapping (*10 *commands + unknown + silence).*

Split	Total N	Active classes	Command samples	Unknown samples	Silence samples	Used for
`train_small`	10,000	10	10,000	0	0	Phases 1–3 train
`valid_small`	2,500	10	2,500	0	0	Phases 1–3 validation
`test_small`	2,500	10	2,500	0	0	Phases 1–3 diagnostics
`train_extended`	38,210	12	18,946	18,946	318	Phase 4 train
`valid_extended`	4,776	12	2,368	2,368	40	Phase 4 validation
`test_extended`	4,776	12	2,368	2,368	40	Phase 4 test

that maps each merged unknown file to its original source label, ensuring no provenance is lost. For the extended dataset configuration, the unknown pool is built from the original labels present under `train/audio` excluding `_background_noise_`, `__unknown__`, and the padded directory. In practice, this means both command and non-command source clips can contribute to synthetic `__unknown__` mixtures, provided the clips in one mixture come from different classes.

Each synthetic sample is formed by blending 2 or 3 one-second clips. Before mixing, each source is mean-centered, RMS-normalized toward a shared target level, and lightly smoothed with a short delayed component to reduce abrupt local transients. The final mixture is then produced by dense short-frame overlap where all sources remain active throughout the full second with slowly varying weights, encouraging spectro-temporal continuity rather than hard segment boundaries.

To avoid regenerating the same example, uniqueness is tracked at the source-file level (sorted source-path tuple), not at the class-combination level. After synthesis, each candidate is passed through an envelope-based smoothness gate (windowed RMS trajectory checks for abrupt jumps and deep internal valleys). Candidates that fail this gate are discarded and regenerated in a retry loop; only accepted candidates are written to the `__unknown__` directory. During extended-split construction, original unknown candidates are first deduplicated by audio-content hash; if this deduplicated pool is smaller than the required unknown quota, synthetic mixtures are added as a top-up until the target unknown count is reached (or generation attempts are exhausted).

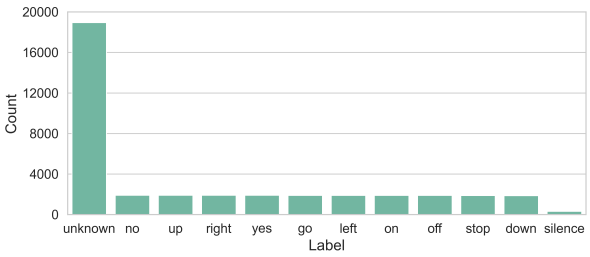

Figure 1: *Class distribution in the extended training set.*

2.2. Audio analysis

In analyzing the raw audio data the focus is on understanding the raw waveforms and their spectral properties. Figure 2 shows an example of a raw audio waveform and its corresponding spectrogram, illustrating the temporal and frequency characteristics of a typical command sample. The spectrogram reveals the energy distribution across frequencies over time, which is crucial for feature extraction strategies. Raw waveforms exhibit high variability in amplitude and temporal structure, while spectrograms provide a more stable representation that captures the essential characteristics of the audio signal. Observing raw waveforms of different classes (Appendix **??**) reveals that amplitude patterns varies, however the main signal begins approximately after 0.4 seconds.

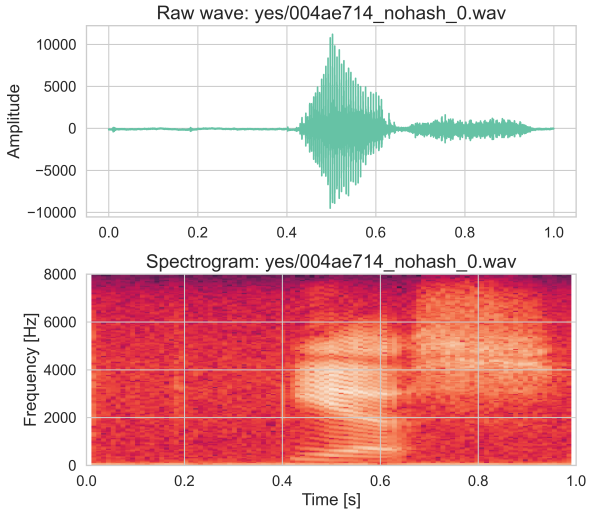

Figure 2: *Raw wave and spectrogram example from train set for "yes".*

Another factor to consider is the average spectral content across different command classes. Figure 3 shows the mean FFT magnitude for each label in the training set, highlighting differences in frequency content between commands. Observation can be made that different commands have their own characteristic which can be exploited by the models to distinguish between them.

3. Experimental Design and MLOps Pipeline

To systematically address our research questions without combinatorial explosion, we filter configurations through a gated, 4-phase funnel. Each phase selects the best setup based on macro-F1 across 3 fixed random seeds, and this configuration is carried forward to the next stage. Table 2 outlines the experiment scope and compute budget. Split usage is fixed by the dataset protocol in Section 2: Phases 1–3 use `small` splits, while Phase 4 uses `extended` splits.

3.1. Uniform Training Protocol

To ensure fair architectural comparison, all models are trained under an identical, deterministic protocol with shared hyper-

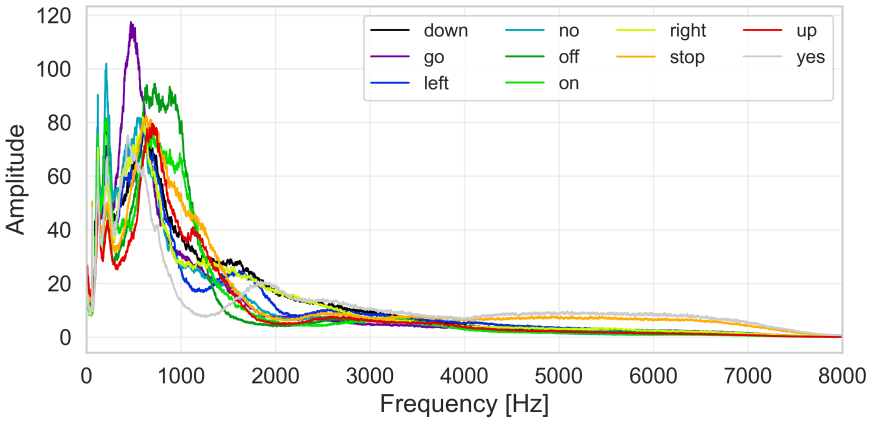

Figure 3: *Mean FFT magnitude per label in the training set.*

Table 2: *Estimated trial executions per experimental phase (one trial may train multiple internal components).*

Phase	Experiment Scope	Configs	Seeds	Total Runs
1	Feature Strategy Ablation (5 features × 2 proxy models)	10	3	30
2	Global Hyperparameter Tuning (best feature × 2 proxies × 4 optim. combos)	8	3	24
3	Inductive Bias Comparison (AST: 8, ConvNeXt: 2, SSAMBA: 8, xLSTM: 8, MLP-Mixer: 4)	30	3	90
4	Non-Command Method Comparison (5 strategies × 3 top models on `__unknown__`/`__silence__`)	15	3	45
Total	**Cumulative Trial Executions**	63		189

parameters and optimization controls. Model training follows AdamW ([5]) with fixed base settings ($\beta_1 = 0.9, \beta_2 = 0.999, \epsilon = 10^{-8}$) and a base learning rate of 3×10^{-4}. Phase 2 searches the AdamW regularization and scheduler settings, and the winning optimizer configuration is then carried forward into Phases 3 and 4. Gradient clipping is applied with a maximum norm threshold of 1.0 to stabilize training on recurrent and attention-based architectures.

The training regimen uses a batch size of 32 across all experiments and model families, with 60 training epochs and a learning-rate scheduler employing cosine annealing with linear warmup (`cosine_annealing_warmup`). A warmup of 5 epochs linearly increases the learning rate from a near-zero start factor of 10^{-3}, followed by cosine decay to a minimum rate of 10^{-6}. Early stopping is enforced with a patience of 10 epochs without macro-F1 improvement on the validation set. Deterministic execution is enabled via fixed seed initialization, and validation is performed every epoch to track model convergence.

3.2. Phase 1: Feature Extraction Strategy (Q1)

Raw audio waveforms are highly inefficient for direct neural network processing due to a lack of semantic meaning at the sample level and high adjacent correlation [6]. We transform the signal into time-frequency representations using the Short-Time Fourier Transform (STFT) [7].

Feature extraction is governed by the time-frequency uncertainty principle (Gabor limit), which bounds resolution such that $\Delta t \cdot \Delta f \geq \frac{1}{4\pi}$. We encapsulate all preprocessing pipelines within PyTorch `nn.Modules` to ensure differentiable, GPU-accelerated computation, evaluating five distinct strategies:

Mel-Spectrogram (Baseline): Emulating human auditory non-linearity via the Mel scale ($m = 2595 \log_{10}(1 + f/700)$) [8], this baseline uses $n_{fft} = 1024$ (64 ms window) and a 10 ms stride. Power spectra are filtered and compressed via an 80 dB-bounded log-amplitude transform to stabilize gradients.

High Temporal Resolution Mel: Shifting the Gabor limit temporally, this setup halves the window and hop sizes (32 ms window; 5 ms stride). The 200 Hz frame rate exposes sub-phonemic transients otherwise smeared by wider windows.

MFCC: Applying a Discrete Cosine Transform (DCT) to log-mel energies projects the signal into the quefrency domain. Retaining 40 coefficients decorrelates channels [9] and slashes the data footprint by $\sim 93\%$.

PCEN: Per-Channel Energy Normalization [10] applies dynamic automatic gain control. It suppresses stationary noise and emphasizes transients [11], enhancing robustness against varied SNRs.

Mel + SpecAugment: Acting as structured input dropout, SpecAugment [12] applies stochastic rectangular masks across time and frequency axes, forcing the model to learn distributed phonetic representations.

To prevent favoring a specific architecture during this phase, we evaluate the pipelines on two proxy baselines: a spatial ConvNeXt model and a sequential xLSTM model. ConvNeXt is implemented with a 1-channel input stem and shared adapter-level shape normalization, which keeps all listed feature variants (including 40×101 MFCC) executable in the current stack. This ensures we do not inadvertently handicap sequential models that might uniquely benefit from high-temporal strides. Table 3 and Figure **??** detail these setups.

3.3. Phases 2 & 3: Architectural Bias and Optimization (Q2)

To isolate structural effects, we train all models entirely from random initialization. Rather than asking which pre-training source transfers best, this design evaluates which architecture learns most effectively from the supervision signal itself.

Phase 2 tunes the global AdamW configuration (Table 4) on the best Phase 1 feature set. Phase 3 then sweeps model-specific hyperparameters (Table 5) using the winning Phase 2 optimizer settings across 5 distinct families:

AST (Audio Spectrogram Transformer): Implemented with Hugging Face `ASTForAudioClassification` (from `transformers`) using `ASTConfig` with hidden size 512, 8 encoder layers, 8 attention heads, and intermediate size 2048 (targeted ˜25M-scale setup in this work). *Key feature / why chosen:* global self-attention from the first layer gives immediate all-to-all context, making AST the explicit "global dependency" reference model [13, 3].

ConvNeXt (Modern CNN): Implemented as `torchvision.models.convnext_tiny` (primary backend), with the first stem convolution adapted to one-channel spectrogram input; `timm convnext_tiny` is kept as backend fallback. *Key feature / why chosen:* strong locality and translation-equivariant inductive bias provide the canonical modern CNN baseline against which global and recurrent models are compared [14, 15].

SSAMBA (Selective State Space): Implemented through `mamba_ssm.Mamba` blocks in a repository wrapper (`MambaHead`), stacked with configuration $d_{\text{model}} = 768$, $d_{\text{state}} = 64$, $d_{\text{conv}} = 4$, expand $= 2$, and 6 layers (targeted ˜25M-scale setup in this work). *Key feature / why chosen:* input-selective state-space updates target long-range sequence modeling with linear-time scaling, representing an efficient alternative to full attention [16, 17].

xLSTM (High-Capacity Recurrence): Implemented with `xlstm.xLSTMBlockStack` in a repository wrapper (`XLSTMHead`), using embedding dimension 704/768 and 8 blocks (alternating mLSTM/sLSTM placements; targeted ˜25M-scale setup). *Key feature / why chosen:* matrix-memory recurrence $C_t \in \mathbb{R}^{d \times d}$ increases recurrent capacity beyond standard LSTM cells, serving as the high-capacity recurrent comparator [18].

MLP-Mixer: Implemented as `timm gmixer_24_224` (official gMLP/Mixer-family model) for the MLP-only baseline. *Key feature / why chosen:* token/channel mixing via dense projections with no explicit convolutional or recurrent prior acts as an architectural "minimal prior" control [19].

The counts in Table 5 are measured on the repository's adapterized backbones, not on the upstream libraries' unmodified reference models. This keeps the comparison honest about the actual code we run while still keeping the backbones in a comparable parameter band; the point is adapter-level parity, not matching an external giant-model budget.

3.4. Phase 4: Non-Command Class Methodology (Q3)

All core models are optimized natively via standard `CrossEntropyLoss` [20]. We intentionally defer imbalance-specific loss shaping and resampling in earlier phases to keep architectural comparisons interpretable. Phases 1–3 use command-only data splits while preserving a shared implementation template for model heads. Phase 4 keeps the selected backbone family and hyperparameters fixed and retrains on `train_extended` for explicit non-command handling. In this sense, "freezing the top 3 architectures" refers to freezing architecture and optimization choices selected upstream. In Phase 4, we test 5 alternatives for handling `__unknown__` and `__silence__`:

1. **Flat multiclass baseline:** single-head 12-class classification.
2. **Sampling control:** weighted random sampling with replacement and fixed target class priors: $p_{\text{unknown}} = 0.25$, $p_{\text{silence}} = 0.15$, and $p_{\text{cmd},k} = 0.06$ for each of the 10 command labels (summing to 1.0).
3. **Loss reweighting:** weighted cross-entropy with elevated penalties on non-command misclassification.
4. **Two-stage detector:** a separate binary gate (command vs. non-command), with command and non-command branch probabilities fused through gate-conditioned composition.
5. **Shared two-head model:** a command head (10 labels) plus a dedicated non-command head (silence vs. unknown).

For sampling control, if class c has n_c training examples, each example from that class receives weight

$$w_c = \frac{p_c}{n_c}.$$

Each epoch draws N samples (with replacement), where N equals the original training-split size. In the implemented Phase 4 loader, inverse-frequency weighting with explicit silence up-weighting is applied as a base balancing mechanism, and strategy-specific controls are layered on top (e.g., strategy 2 priors and strategy 3 loss weights). Unknown-sample synthesis remains conditionally enabled: additional synthetic unknown samples are generated only when empirical unknown support falls below the mean support of the command classes.

To keep comparisons fair, all methods reuse identical data splits and 3 fixed seeds (0, 42, and 2003). Model selection follows a strict gate: any method that degrades core-command macro-F1 by more than $\delta = 1.0$ percentage point relative to its corresponding Phase 3 command baseline is rejected. Among remaining candidates, we maximize non-command macro-F1,

$$\text{Macro-F1}_{\text{NC}} = \frac{\text{F1}_{\text{unknown}} + \text{F1}_{\text{silence}}}{2},$$

Table 3: *Feature extraction strategies compared in Phase* 1.

Strategy	n_fft	hop_length	Window	Hop	Frames	Bins	Approx. input size (1 s)	Augmentation
Mel (baseline)	1024	160	64 ms	10 ms	101	128 mels	12,928 (128 × 101)	none
Mel (hi-temporal)	512	80	32 ms	5 ms	201	128 mels	25,728 (128 × 201)	none
MFCC	1024	160	64 ms	10 ms	101	40 MFCCs	4,040 (40 × 101)	none
PCEN	1024	160	64 ms	10 ms	101	128 mels	12,928 (128 × 101)	none
Mel + SpecAugment	1024	160	64 ms	10 ms	101	128 mels	12,928 (128 × 101)	freq+time masking

Table 4: *Global optimization hyperparameters (Phase* 2*).*

Parameter	**Values**	**Purpose**
AdamW learning rate	3×10^{-4}	Fixed base step size reused by later phases
AdamW β_1, β_2	$(0.9, 0.999)$	Fixed first- and second-moment coefficients
AdamW ϵ	10^{-8}	Fixed numerical stability constant
Weight decay	$10^{-2}, 10^{-1}$	Swept AdamW regularization strength
LR scheduler	Cosine Annealing with Warmup / Reduce LR on Plateau	Swept scheduler family; selected schedule is reused later

and use silence false-trigger rate and unknown-to-command leakage as tie-breakers.

The outputs of Phase 4 define the final deployable systems and are the only systems advanced to held-out test-set validation. Strategy 4 is executed as one trial configuration that internally trains separate gate, command, and non-command components; Table 2 reports trial-level counts rather than internal component fits. We evaluate 3 inference flows: (i) single-stage multiclass flow (input → one model → class), (ii) two-stage detector flow (input → binary gate and branch experts → gate-weighted probability fusion), and (iii) shared-backbone two-head flow (input → shared encoder → command head and non-command head, with a fixed decision rule).

4. Strict Reproducibility and Training Control

To account for hardware-level non-determinism during GPU acceleration, we execute every experiment three separate times using fixed starting points (seeds 0, 42, and 2003) and report aggregated results (mean ± standard deviation).

All experiments are run on a single Apple MacBook Pro with an M4 Pro chip, 48 GB unified memory, and a 1024 GB SSD. This hardware budget motivates the use of `train_small` and `valid_small` in Phases 1–3.

To stabilize compute and comparisons in Phases 1–3, the small splits are fixed to exactly 1,000/250/250 samples per command class for train/validation/test, respectively.

We employ AdamW [5] to explicitly decouple weight decay from adaptive gradient scaling, and rely on Kaiming normal initialization [21]. To guarantee that experiments can be perfectly duplicated, we enforce deterministic data loading and automatically generate a `ReproducibilityReport` via MLflow [22] for every run. This report logs all environmental conditions, library versions, and the precise state of the codebase.

5. Results

Final test reporting covers all completed Phase 4 trials, since this stage determines deployment-relevant behavior for `__unknown__` and `__silence__`. After the Phase 4 sweep completes, each trial is evaluated once on the held-out test set. We report per-class precision, recall, and F1, together with command macro-F1 and non-command macro-F1.

Beyond single-model results, we evaluate all ensemble combinations across the 3 Phase 3-derived backbones within each fixed Phase 4 method. Denoting model outputs by M_1, M_2, M_3, we test all non-empty subsets (i.e., $2^3 - 1 = 7$ combinations) and aggregate class probabilities by simple averaging under a common within-method calibration rule. Cross-method ensembles are not considered; each ensemble is built from models produced by a single Phase 4 strategy. These ensemble checks are inference-only and therefore excluded from the cumulative training execution counts in Table 2.

6. Conclusion

This work defines a reproducible, 4-phase methodology for studying KWS under controlled conditions, from feature selection and architecture comparison to explicit non-command handling. Final systems are selected in Phase 4, and the evaluation plan includes both single-model testing and within-method ensemble testing across the 3 best Phase 3 backbones. At this stage, the paper serves as a protocol and analysis blueprint rather than a source of empirical claims. Quantitative conclusions about architecture quality and boundary-class behavior will be drawn only after full Phase 4 selection and held-out test evaluation are completed.

7. Future Work

While this study establishes a strictly controlled baseline for evaluating KWS architectures, several avenues remain for future exploration. First, because our methodology isolates architectural inductive biases by training entirely from random initialization, future work could compare these baselines against pre-trained variants to quantify the performance gains afforded by transfer learning. Second, although our data preprocessing enforces a uniform one-second temporal support, adapting the top-performing architectures to process variable-length inputs or continuous, untrimmed audio streams would better simulate true always-on deployment conditions. Third, while the synthesis of the `__unknown__` class is currently limited to fixed extended splits, deploying our proposed synthesis pipeline as a dynamic, on-the-fly augmentation mechanism could further harden models against out-of-vocabulary spoken utterances. Finally, our current ensemble evaluation is restricted to within-method combinations; future research could explore cross-method ensembles to determine if heterogeneous non-command handling strategies yield superior robustness against false triggers.

Table 5: *Model-specific hyperparameter sweeps (Phase* 3*), including architecture type and parameter count from the repo-specific adapters used in this work (default non-pretrained configuration,* 12 *output classes).*

Model	Arch. type	Total params	Sweep Space
AST	Transformer (global attention)	25,416,204	Dropout $\{0.1, 0.5\}$; Head {linear, MLP(256)}; Pos. Emb. {interp., learned}; fixed size: hidden $= 512$, layers $= 8$, heads $= 8$, intermediate $= 2048$ [3]
ConvNeXt	CNN (local spatial hierarchy)	27,826,284	Stoch. Depth $\{0.0, 0.2\}$; Kernel size $\{7 \times 7\}$ [15]
SSAMBA	State-space sequence model	23,963,148	Pooling {mean, max}; CLS {yes, no}; Stride $\{10\text{ms}, 5\text{ms}\}$; fixed size: $d_{\text{model}} = 768$, $d_{\text{state}} = 64$, expand $= 2$, layers $= 6$ [16, 17]
xLSTM	Recurrent sequence model	24,259,180	Dim $d \in \{704, 768\}$; State Reset {T, F}; Output {final, mean}; fixed depth: blocks $= 8$ [18]
MLP-Mixer	MLP-only token/channel mixer	24,144,108	Dropout $\{0.0, 0.2\}$; Head L2 Norm {yes, no}; default timm variant: `gmixer_24_224` [19]

8. References

[1] T. N. Sainath and C. Parada, "Convolutional neural networks for small-footprint keyword spotting," in *Sixteenth annual conference of the international speech communication association*, 2015.

[2] Y. Zhang, N. Suda, L. Lai, and V. Chandra, "Hello edge: Keyword spotting on microcontrollers," *arXiv preprint arXiv:1711.07128*, 2017.

[3] Y. Gong, Y.-A. Chung, and J. Glass, "Ast: Audio spectrogram transformer," in *Interspeech 2021*, 2021, pp. 571–575.

[4] P. Warden, "Speech commands: A dataset for limited-vocabulary speech recognition," *arXiv preprint arXiv:1804.03209*, 2018.

[5] I. Loshchilov and F. Hutter, "Decoupled weight decay regularization," in *International Conference on Learning Representations*, 2019.

[6] H. Purwins, B. Li, T. Virtanen, J. Schlüter, S.-Y. Chang, and T. Sainath, "Deep learning for audio signal processing," *IEEE Journal of Selected Topics in Signal Processing*, vol. 13, no. 2, pp. 206–219, 2019.

[7] J. B. Allen, "Short term spectral analysis, synthesis, and modification by discrete fourier transform," *IEEE Transactions on Acoustics, Speech, and Signal Processing*, vol. 25, no. 3, pp. 235–238, 1977.

[8] S. S. Stevens, J. Volkmann, and E. B. Newman, "A scale for the measurement of the psychological magnitude pitch," *The Journal of the Acoustical Society of America*, vol. 8, no. 3, pp. 185–190, 1937.

[9] S. Davis and P. Mermelstein, "Comparison of parametric representations for monosyllabic word recognition in continuously spoken sentences," *IEEE transactions on acoustics, speech, and signal processing*, vol. 28, no. 4, pp. 357–366, 1980.

[10] Y. Wang, P. Getreuer, T. Hughes, R. F. Lyon, and R. A. Saurous, "Trainable frontend for robust far-field keyword spotting," in *2017 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*. IEEE, 2017, pp. 5670–5674.

[11] V. Lostanlen, J. Salamon, M. Cartwright, B. McFee, A. Farnsworth, S. Kelling, and J. P. Bello, "Per-channel energy normalization: Why and how," *IEEE Signal Processing Letters*, vol. 26, no. 1, pp. 39–43, 2019.

[12] D. S. Park, W. Chan, Y. Zhang, C.-C. Chiu, B. Zoph, E. D. Cubuk, and Q. V. Le, "Specaugment: A simple data augmentation method for automatic speech recognition," in *Interspeech 2019*, 2019, pp. 2613–2617.

[13] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser, and I. Polosukhin, "Attention is all you need," *Advances in neural information processing systems*, vol. 30, 2017.

[14] Y. LeCun, Y. Bengio *et al.*, "Convolutional networks for images, speech, and time series," *The handbook of brain theory and neural networks*, vol. 3361, no. 10, p. 1995, 1995.

[15] Z. Liu, H. Mao, C.-Y. Wu, C. Feichtenhofer, T. Darrell, and S. Xie, "A convnet for the 2020s," in *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, 2022, pp. 11 976–11 986.

[16] A. Gu and T. Dao, "Mamba: Linear-time sequence modeling with selective state spaces," *arXiv preprint arXiv:2312.00752*, 2023.

[17] S. Shams, S. S. Dindar, X. Jiang, and N. Mesgarani, "Ssamba: Self-supervised audio representation learning with mamba state space model," *arXiv preprint arXiv:2405.11831*, 2024.

[18] M. Beck, K. Pöppel, M. Spanring, A. Auer, O. Prudnikova, M. Kopp, G. Klambauer, J. Brandstetter, and S. Hochreiter, "xlstm: Extended long short-term memory," *arXiv preprint arXiv:2405.04517*, 2024.

[19] I. O. Tolstikhin, N. Houlsby, A. Kolesnikov, L. Beyer, X. Zhai, T. Unterthiner, J. Yung, A. Steiner, D. Keysers, J. Uszkoreit, M. Lucic, and A. Dosovitskiy, "Mlp-mixer: An all-mlp architecture for vision," in *Advances in Neural Information Processing Systems*, vol. 34, 2021, pp. 24 261–24 272.

[20] I. Goodfellow, Y. Bengio, and A. Courville, *Deep Learning*. MIT Press, 2016. [Online]. Available: http://www.deeplearningbook.org

[21] K. He, X. Zhang, S. Ren, and J. Sun, "Delving deep into rectifiers: Surpassing human-level performance on imagenet classification," in *Proceedings of the IEEE international conference on computer vision*, 2015, pp. 1026–1034.

[22] M. Zaharia, A. Chen, A. Davidson, A. Ghodsi, S. A. Hong, A. Konwinski, S. Murching, T. Nykodym, P. Ogilvie, M. Parkhe *et al.*, "Accelerating the machine learning lifecycle with mlflow," in *IEEE Data Eng. Bull.*, vol. 41, no. 4, 2018, pp. 39–45.

A. Additional Analyses and Visualizations

Figures 4, 5, and 6 provide additional waveform–spectrogram examples for the labels *down*, *no*, and *up*, respectively. These supplementary plots complement the main text by illustrating cross-class variability in temporal envelope shape, onset location, and spectral energy concentration. Together, they support the motivation for using time-frequency features and for comparing architectures with different inductive biases for local versus global temporal-spectral structure.

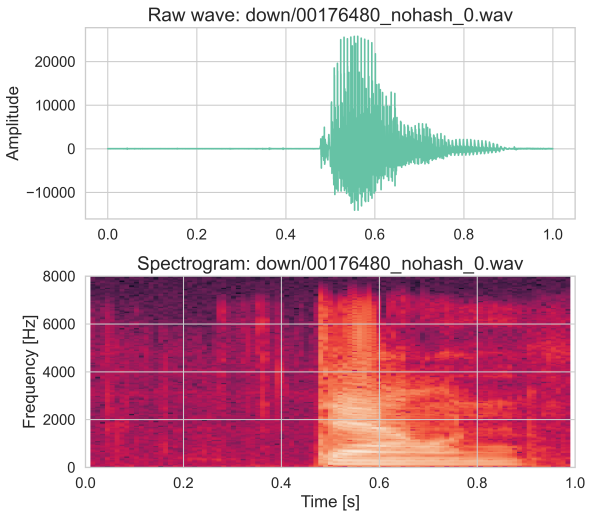

Figure 4: *Raw wave and spectrogram example from train set for "down".*

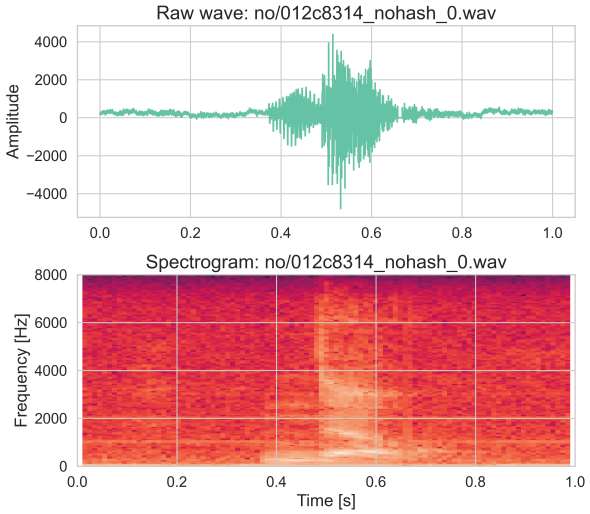

Figure 5: *Raw wave and spectrogram example from train set for "no".*

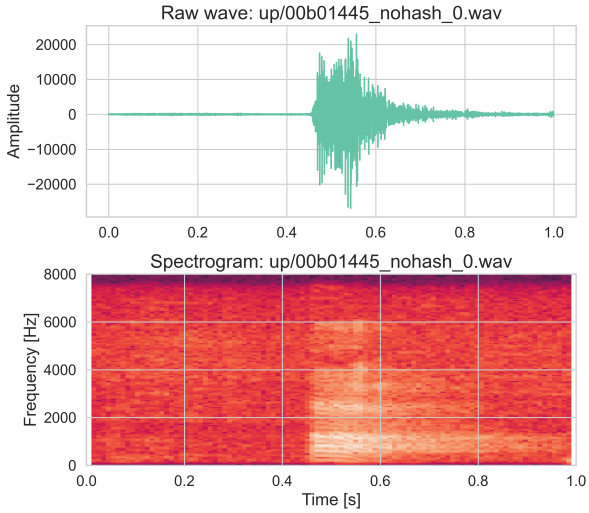

Figure 6: *Raw wave and spectrogram example from train set for "up".*